



MATH1222 INTRODUCTION AUX PROCESSUS STOCHASTIQUES
RAPPORT

Méthodes de Monte Carlo par chaînes de Markov pour la détection de motifs en bioinformatique

LOUIS PEQUET s2300798 & TOM DENIS s2301633

ACADEMIC YEAR 2025-2026

1 Partie 1 : Chaîne de Markov et échantillonnage de Gibbs

1.1 Chaînes de Markov

Question 1

Soit la fonction de vraisemblance :

$$L(P) = P(X_0) \prod_{t=0}^{T-1} P(X_{t+1}|X_t)$$

Si $n_{i,j}$ est le nombre de fois que la transition de l'état i vers l'état j apparaît dans la séquence, alors :

$$L(P) \propto \prod_{i=1}^4 \prod_{j=1}^4 (P_{i,j})^{n_{i,j}}$$

Et en prenant le logarithme de la fonction de vraisemblance on obtient :

$$\ell(P) = \ln L(P) \propto \sum_{i=1}^4 \sum_{j=1}^4 n_{i,j} \ln(P_{i,j})$$

Pour une ligne spécifique i , on définit le Lagrangien $\mathcal{L}_i$ avec un multiplicateur λ :

$$\mathcal{L}_i = \sum_{j=1}^4 n_{i,j} \ln(P_{i,j}) - \lambda \left(\sum_{j=1}^4 P_{i,j} - 1 \right)$$

Pour trouver le maximum, nous avons que :

$$\frac{\partial \mathcal{L}_i}{\partial P_{i,j}} = \frac{n_{i,j}}{P_{i,j}} - \lambda = 0$$

Ce qui nous donne :

$$P_{i,j} = \frac{n_{i,j}}{\lambda}$$

Sachant que dans la matrice de transition la somme des éléments d'une ligne est égal à 1, on peut remplacer $P_{i,j}$ et déterminer λ :

$$\begin{aligned} \sum_{j=1}^4 P_{i,j} &= 1 \\ \sum_{j=1}^4 \left(\frac{n_{i,j}}{\lambda} \right) &= 1 \\ \lambda &= \sum_{j=1}^4 n_{i,j} \end{aligned}$$

Ce qui nous donne enfin :

$$P_{i,j} = \frac{n_{i,j}}{\sum_{k=1}^4 n_{i,k}}$$

En utilisant la formule ci-dessus, nous calculons la matrice de transition (P) de la chaîne de Markov modélisant la première séquence d'ADN fournis *seq1* (voir code python). La figure 1 ci-dessous représente le diagramme d'états de la chaîne correspondante.

$$P = \begin{matrix} & \begin{matrix} A & C & G & T \end{matrix} \\ \begin{matrix} A \\ C \\ G \\ T \end{matrix} & \begin{pmatrix} 0.0972 & 0.5052 & 0.2778 & 0.1198 \\ 0.2532 & 0.2461 & 0.2618 & 0.2389 \\ 0.3110 & 0.4928 & 0.1077 & 0.0885 \\ 0.6961 & 0.0980 & 0.0980 & 0.1078 \end{pmatrix} \end{matrix} \quad (1)$$

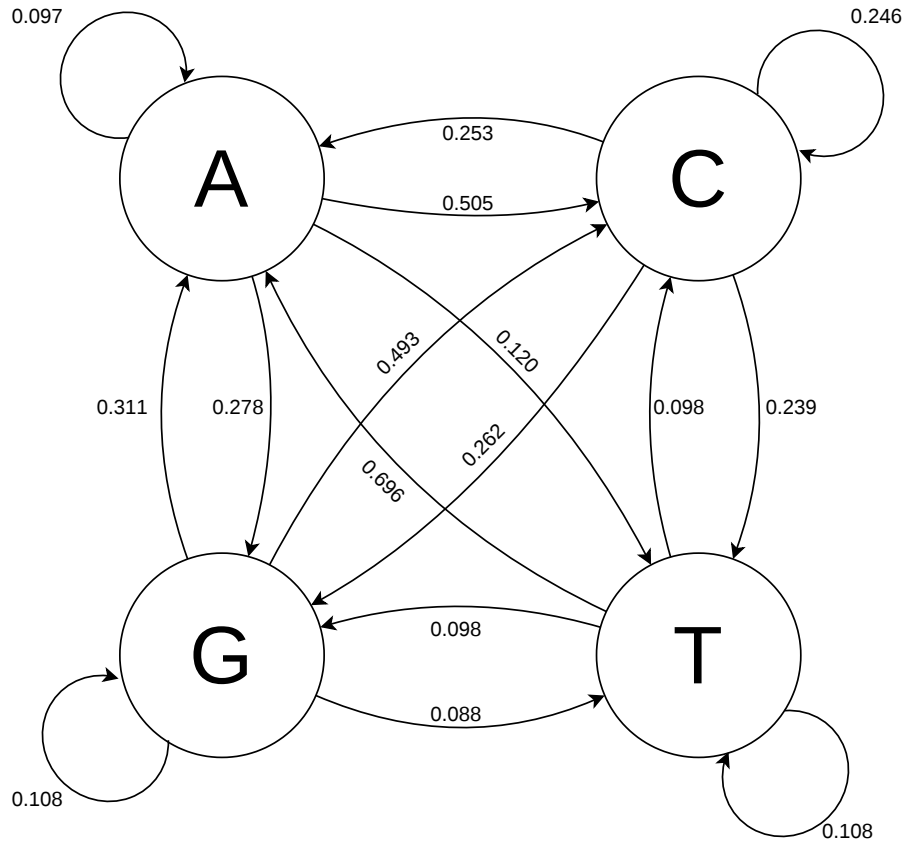


FIGURE 1 – Diagramme d'états

Question 2

Tout d'abord, nous présentons ci-dessous les puissances successives de la matrice de transition P . Nous observons qu'à mesure que l'exposant croît, la matrice converge vers une matrice limite dont la caractéristique principale est l'identité de ses lignes. Celles-ci correspondent au vecteur de probabilités stationnaires π , tel que $\pi P = \pi$ (avec une précision de quatre décimales atteinte dès la dixième puissance).

$$\begin{aligned}
P^1 &= \begin{pmatrix} 0.0972 & 0.5052 & 0.2778 & 0.1198 \\ 0.2532 & 0.2461 & 0.2618 & 0.2389 \\ 0.3110 & 0.4928 & 0.1077 & 0.0885 \\ 0.6961 & 0.0980 & 0.0980 & 0.1078 \end{pmatrix} & P^2 &= \begin{pmatrix} 0.3072 & 0.3221 & 0.2009 & 0.1699 \\ 0.3347 & 0.3409 & 0.1864 & 0.1381 \\ 0.2501 & 0.3401 & 0.2357 & 0.1741 \\ 0.1981 & 0.4347 & 0.2401 & 0.1271 \end{pmatrix} \\
P^3 &= \begin{pmatrix} 0.2921 & 0.3501 & 0.2079 & 0.1498 \\ 0.2729 & 0.3583 & 0.2158 & 0.1529 \\ 0.3049 & 0.3433 & 0.2010 & 0.1509 \\ 0.2925 & 0.3378 & 0.2071 & 0.1625 \end{pmatrix} & P^4 &= \begin{pmatrix} 0.2860 & 0.3509 & 0.2099 & 0.1532 \\ 0.2908 & 0.3474 & 0.2079 & 0.1539 \\ 0.2841 & 0.3523 & 0.2110 & 0.1526 \\ 0.2915 & 0.3489 & 0.2079 & 0.1516 \end{pmatrix} \\
P^5 &= \begin{pmatrix} 0.2886 & 0.3493 & 0.2089 & 0.1532 \\ 0.2880 & 0.3499 & 0.2092 & 0.1528 \\ 0.2887 & 0.3492 & 0.2088 & 0.1533 \\ 0.2869 & 0.3505 & 0.2096 & 0.1530 \end{pmatrix} & P^{10} &= \begin{pmatrix} 0.2881 & 0.3497 & 0.2091 & 0.1531 \\ 0.2881 & 0.3497 & 0.2091 & 0.1531 \\ 0.2881 & 0.3497 & 0.2091 & 0.1531 \\ 0.2881 & 0.3497 & 0.2091 & 0.1531 \end{pmatrix} \\
P^{15} &= \begin{pmatrix} 0.2881 & 0.3497 & 0.2091 & 0.1531 \\ 0.2881 & 0.3497 & 0.2091 & 0.1531 \\ 0.2881 & 0.3497 & 0.2091 & 0.1531 \\ 0.2881 & 0.3497 & 0.2091 & 0.1531 \end{pmatrix}
\end{aligned}$$

Ensuite, en observant les deux graphiques ci-dessous, nous constatons tout d'abord que les probabilités $P(X_t = i)$ convergent vers les mêmes valeurs limites et ce, malgré les différentes distributions initiales. Ce comportement est caractéristique des chaînes de Markov régulières, qui admettent une unique distribution stationnaire indépendante de l'état initial. Dans notre cas de figure, la convergence est pratiquement atteinte dès $t \approx 5$. En effet, l'impact de la loi initiale est visible uniquement durant les premières transitions ($t \in \{0, 1, 2\}$).

Par ailleurs, nous remarquons une certaine similitude entre le graphique de gauche (initialisation uniforme) et celui de droite (initialisation en C) avec un décalage d'une unité de temps. Cette observation s'explique par la structure de la matrice P . En effet, les probabilités de transition depuis l'état C (correspondant à la deuxième ligne de la matrice) sont distribuées de manière quasi-uniforme entre les quatre nucléotides (0.253, 0.246, 0.262, 0.239). Ainsi, dès la première itération ($t = 1$), la distribution obtenue en partant de l'état C est numériquement très proche d'une loi uniforme, ramenant le système à une configuration similaire à l'état initial du premier graphique.

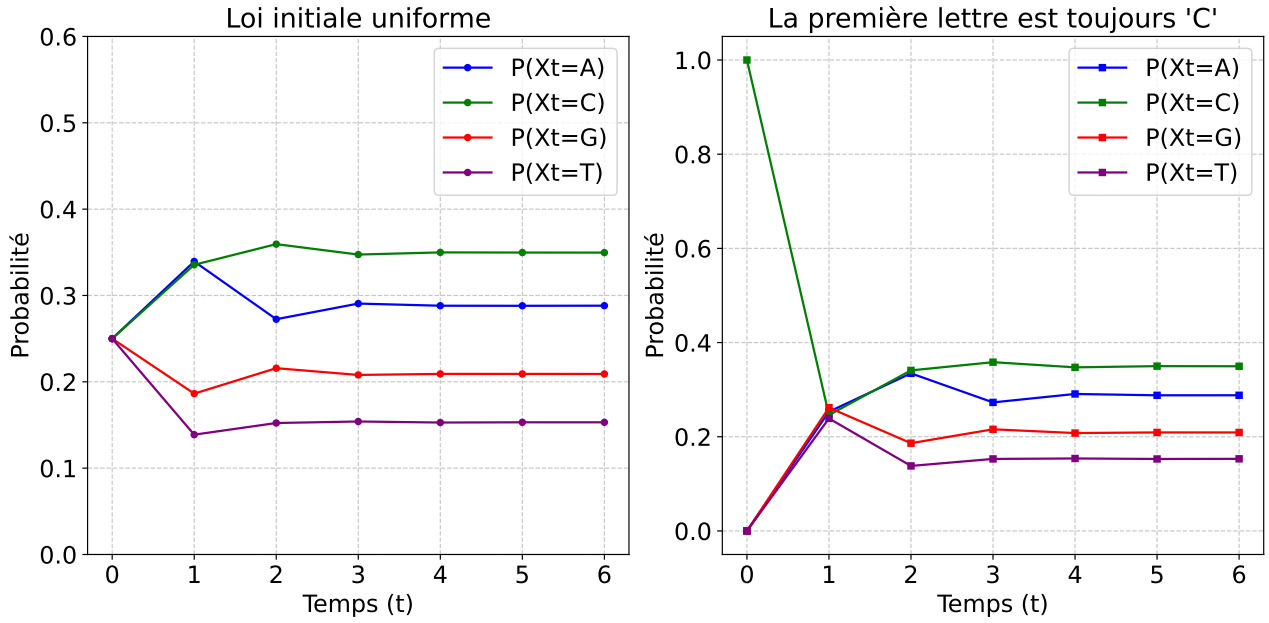


FIGURE 2 – Graphiques représentant l'évolution de la probabilité d'obtenir chaque nucléotide en fonction du nombre de transitions effectuées selon la distribution initiale.

Question 3

Puisque tous les éléments de la matrice de transition sont strictement positif alors la chaîne est régulière (irréductible et apériodique). Ainsi puisque la chaîne est régulière, elle admet alors une distribution stationnaire et vérifie cette équation :

$$\pi_{\infty} P = \pi_{\infty}$$

$$(\pi_{\infty}^1 \quad \pi_{\infty}^2 \quad \pi_{\infty}^3 \quad \pi_{\infty}^4) \begin{pmatrix} 0.0972 & 0.5052 & 0.2778 & 0.1198 \\ 0.2532 & 0.2461 & 0.2618 & 0.2389 \\ 0.3110 & 0.4928 & 0.1077 & 0.0885 \\ 0.6961 & 0.0980 & 0.0980 & 0.1078 \end{pmatrix} = (\pi_{\infty}^1 \quad \pi_{\infty}^2 \quad \pi_{\infty}^3 \quad \pi_{\infty}^4)$$

sous la contrainte :

$$\pi_{\infty}^1 + \pi_{\infty}^2 + \pi_{\infty}^3 + \pi_{\infty}^4 = 1$$

On obtient donc ce système d'équations :

$$\begin{cases} 0.0972\pi_{\infty}^1 + 0.5052\pi_{\infty}^2 + 0.2778\pi_{\infty}^3 + 0.1198\pi_{\infty}^4 = \pi_{\infty}^1 \\ 0.2532\pi_{\infty}^1 + 0.2461\pi_{\infty}^2 + 0.2618\pi_{\infty}^3 + 0.2389\pi_{\infty}^4 = \pi_{\infty}^2 \\ 0.311\pi_{\infty}^1 + 0.4928\pi_{\infty}^2 + 0.1077\pi_{\infty}^3 + 0.0885\pi_{\infty}^4 = \pi_{\infty}^3 \\ 0.6961\pi_{\infty}^1 + 0.098\pi_{\infty}^2 + 0.0980\pi_{\infty}^3 + 0.1078\pi_{\infty}^4 = \pi_{\infty}^4 \\ \pi_{\infty}^1 + \pi_{\infty}^2 + \pi_{\infty}^3 + \pi_{\infty}^4 = 1 \end{cases}$$

En résolvant ce système on obtient :

$$\pi_{\infty} = (0.2881 \quad 0.3497 \quad 0.2091 \quad 0.153)$$

Note : Nous retrouvons bien les mêmes valeurs que celles calculées dans la section précédente via le calcul des puissances de la matrices P.

Question 4

En observant le graphique ci-dessous, nous constatons que, pour les séquences de petite longueur, les fréquences d'apparition de chaque lettre sont assez variables et fluctuent d'une simulation à l'autre. Cependant, pour les séquences de plus grande longueur, les fréquences d'apparition convergent vers les valeurs de la distribution stationnaire établies précédemment. Nous en concluons que la distribution stationnaire déterminée précédemment est correcte et validée expérimentalement.

De plus, nous avons arbitrairement choisi de générer la séquence en commençant par le nucléotide T. Cependant, comme évoqué dans la question 2 de ce rapport, ce graphique montre à nouveau que ce choix n'a d'influence que pour de très petites séquences, et que les résultats pour les grandes séquences en sont indépendants.

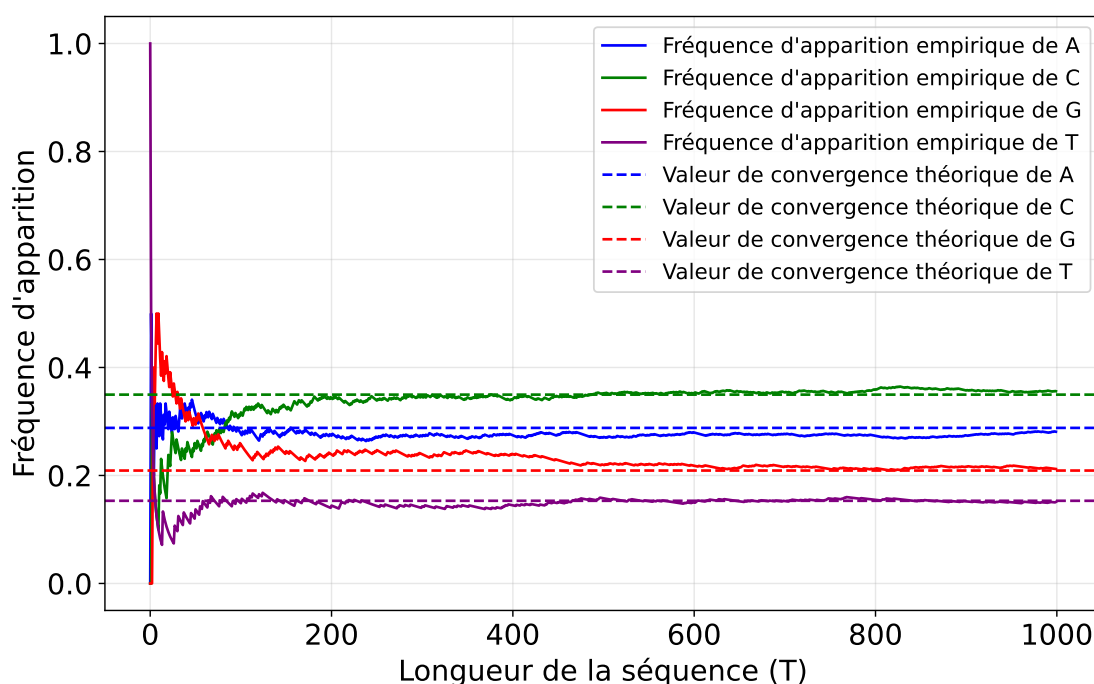


FIGURE 3 – Graphique représentant l'évolution des fréquences d'apparition des quatre nucléotides dans une séquence générée par notre chaîne de Markov en fonction de sa longueur

Question 5

Les fréquences d'apparition des quatre lettres dans les trois séquences fournies sont résumées dans le tableau ci-dessous :

Séquence	A	C	G	T
seq1	0.2885	0.3495	0.2090	0.1530
seq2	0.3060	0.3415	0.1970	0.1555
seq3	0.2845	0.3605	0.1985	0.1565

Ensuite, afin de calculer les probabilités que les trois séquences aient été générées par la chaîne de Markov estimée à partir de la première, nous avons utilisé les valeurs de log-vraisemblance. Cette approche permet d'éviter la manipulation de valeurs extrêmement proches de zéro. Les résultats obtenus sont les suivants :

Séquence	Log-vraisemblance
seq1	-2427
seq2	-2971
seq3	-2400

Nous observons que les fréquences d'apparition des lettres dans les trois séquences sont très proches de la distribution stationnaire établie précédemment. En revanche, l'analyse des log-vraisemblances révèle des différences. En effet, les séquences 1 et 3 présentent des valeurs relativement proches, ce qui suggère qu'elles ont probablement été générées par la chaîne de Markov considérée alors que la séquence 2, quant à elle, possède une valeur significativement plus faible indiquant qu'elle est moins compatible avec ce modèle. Nous en concluons que la connaissance des fréquences d'apparition ne suffit pas à déterminer si des séquences ont été générées, ou non, par une même chaîne de Markov.

Par ailleurs, il est important de noter que cette méthode de comparaison n'est pertinente que parce que les trois séquences ont la même longueur. En effet, plus une séquence est longue, plus sa probabilité de génération est faible, car chaque lettre supplémentaire multiplie la probabilité par un facteur compris entre 0 et 1.

1.2 Échantillonnage de Gibbs

Question 1

Soit p_{ij} la probabilité de transition de l'état y_i vers l'état y_j , définie par :

$$p(Y) = p_{ij} = P(Y^{t+1} = y_j | Y^t = y_i)$$

On marginalise par rapport à la variable aléatoire I , qui représente l'indice de la composante mise à jour. Dans l'algorithme de Gibbs, I suit une loi uniforme discrète sur $\{1, \dots, N\}$.

$$p_{ij} = \sum_{k=1}^N P(Y^{t+1} = y_j | Y^t = y_i, I = k) P(I = k)$$

Puisque les états y_i et y_j diffèrent uniquement à la position c , la probabilité de transition est nulle si l'algorithme choisit un indice $k \neq c$. Il ne reste donc que le terme où $I = c$, avec $P(I = c) = \frac{1}{N}$. En introduisant la notation y_{i-c} pour désigner toutes les composantes de l'état y_i sauf celle en c , l'algorithme de Gibbs nous donne :

$$p_{ij} = \frac{1}{N} P(Y^{t+1} = y_{j_c} | Y^t = y_{i-c})$$

Via la loi de Bayes, on obtient :

$$\begin{aligned} p_{ij} &= \frac{1}{N} \frac{P(y_{j_c}, y_{i-c})}{P(y_{i-c})} \\ &= \frac{1}{N} \frac{P(y_j)}{P(y_{i-c})} \end{aligned}$$

En injectant ce résultat dans la condition de balance :

$$\begin{aligned} p(y_i) p_{ij} &= p(y_j) p_{ji} \\ p(y_i) \frac{1}{N} \frac{P(y_j)}{P(y_{i-c})} &= p(y_j) \frac{1}{N} \frac{P(y_i)}{P(y_{j-c})} \\ P(y_{i-c}) &= P(y_{j-c}) \end{aligned}$$

Ce qui montre que la condition de balance est bien respectée. Pour que l'algorithme fonctionne la chaîne doit être régulière, c'est-à-dire irréductible et apériodique.

Question 2

a) Une distribution conjointe sur les deux variables Y_1 et Y_2 prenant la forme d'une matrice $Q_Y \in \mathbb{R}^{4 \times 4}$ telle que $[Q_Y]_{i,j} = P(Y_1 = i, Y_2 = j)$ de sorte que l'algorithme de Gibbs puisse être utilisé pour échantillonner selon cette distribution peut être définie comme :

$$Q_Y = \begin{pmatrix} \frac{1}{16} & \frac{1}{16} & \frac{1}{16} & \frac{1}{16} \\ \frac{1}{16} & \frac{1}{16} & \frac{1}{16} & \frac{1}{16} \\ \frac{1}{16} & \frac{1}{16} & \frac{1}{16} & \frac{1}{16} \\ \frac{1}{16} & \frac{1}{16} & \frac{1}{16} & \frac{1}{16} \end{pmatrix}$$

Cette matrice est bien une distribution puisque la somme des éléments est égale à 1. Puisque chaque élément de la matrice est strictement positif, alors n'importe quel état peut être atteint depuis n'importe quel autre état donc la chaîne de Markov est bien irréductible. Et puisque les éléments diagonaux de la matrice sont strictement positifs alors il existe un circuit de longueur 1 pour chaque état donc la chaîne de Markov est bien apériodique.

b) En appliquant l'algorithme de Gibbs pour générer une chaîne de 50000 échantillons nous obtenons ces résultats :

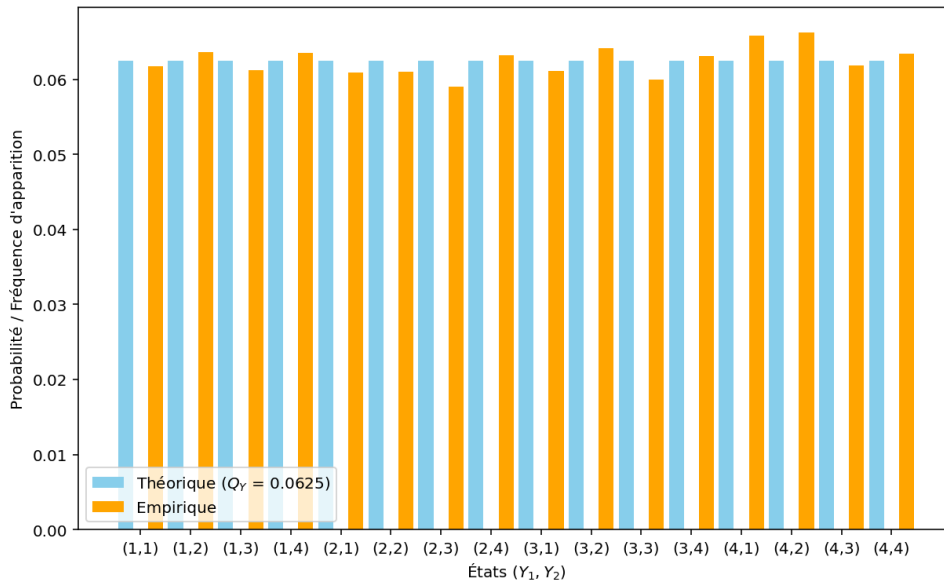


FIGURE 4 – Comparaison des fréquences d'apparition empirique et de la distribution théorique

On peut observer que les fréquences d'apparition tendent vers la distribution théorique Q_Y . Ceci illustre le fait que l'algorithme de Gibbs construit bien une chaîne qui aura une distribution stationnaire donnée.

c) En répétant la même question qu'au point précédent mais en alternant l'échantillonnage de Y_1 puis de Y_2 , on obtient ces résultats :

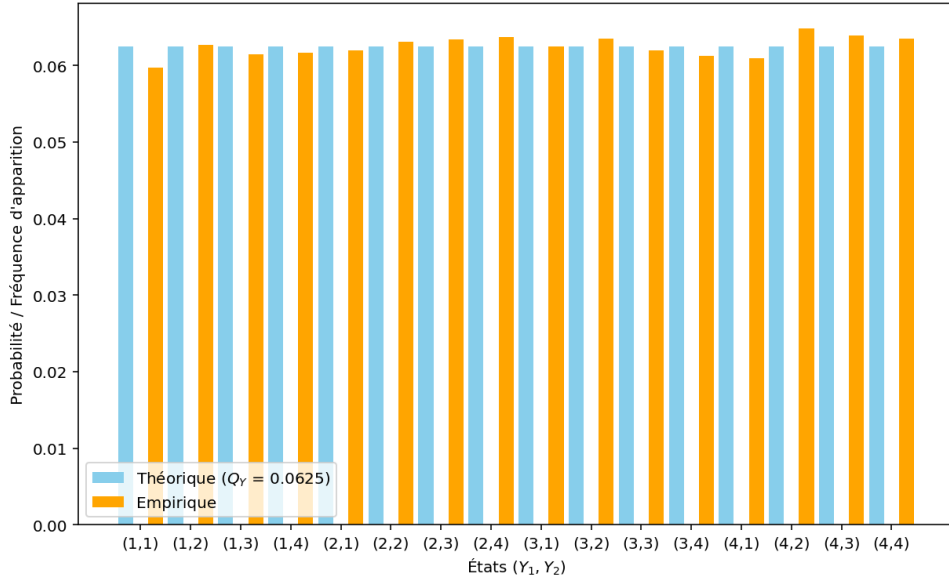


FIGURE 5 – Comparaison des fréquences d'apparition empirique pour un échantillonnage alterné et de la distribution théorique

On peut observer le même phénomène qu'au point précédent, c'est-à-dire que les fréquences d'apparition tendent vers la distribution théorique Q_Y . Le fait que nous obtenions les mêmes résultats illustre que l'algorithme de Gibbs donne la même distribution stationnaire peu importe l'ordre d'échantillonnage.

2 Deuxième partie : détection de motifs dans des séquences d'ADN

2.1 Dérivations mathématiques

Dans un premier temps nous allons déterminer les équations dans le cas où le modèle Φ est une multinomiale. Pour déterminer $p(\Theta, \mathcal{A} | S, \Phi)$, on doit d'abord déterminer la probabilité d'obtenir une certaine séquence sachant les paramètres des modèles du motif et des séquences en dehors du motif :

$$\begin{aligned}
p(S, \mathcal{A} | \Theta, \Phi) &= p(S | \mathcal{A}, \Theta, \Phi) p(\mathcal{A} | \Theta, \Phi) \\
&= p(S | \mathcal{A}, \Theta, \Phi) p(\mathcal{A}) \\
&= \prod_{k=1}^N p(S_k | A_k = a_k, \Theta, \Phi) p(A_k = a_k) \\
&= \prod_{k=1}^N \left(\prod_{i=1}^{a_k-1} \Phi[S_k[i]] \right) \left(\prod_{i=a_k}^{a_k+W-1} \Theta[S_k[i], i - a_k] \right) \left(\prod_{i=a_k+W}^{|S_k|} \Phi[S_k[i]] \right) \frac{1}{|S_k| - W + 1}
\end{aligned}$$

où,

- $\prod_{i=1}^{a_k-1} \Phi[S_k[i]]$ est la probabilité d'obtenir cette séquence avant le motif.
- $\prod_{i=a_k}^{a_k+W-1} \Theta[S_k[i], i - a_k]$ est la probabilité d'obtenir cette séquence comme motif.
- $\prod_{i=a_k+W}^{|S_k|} \Phi[S_k[i]]$ est la probabilité d'obtenir cette séquence après le motif.
- $p(A_k = a_k)$ est la probabilité que la position du premier nucléotide du motif soit a_k . Où A_k est une variable aléatoire discrète distribuée uniformément de 1 à $|S_k| - W + 1$

Ici nous avons fait l'hypothèse que les séquences S_k étaient indépendantes et que la variable aléatoire discrète $\mathcal{A}$ est indépendante de Θ et Φ .

Ensuite, on peut calculer la probabilité *a posteriori* de Θ :

$$\begin{aligned} p(\Theta|\mathcal{A}, \mathcal{S}, \Phi) &= \frac{p(\Theta, \mathcal{A}, \mathcal{S}, \Phi)}{p(\mathcal{A}, \mathcal{S}, \Phi)} \\ &= \frac{p(\mathcal{S}, \mathcal{A}|\Theta, \Phi)p(\Theta|\Phi)p(\Phi)}{p(\mathcal{A}, \mathcal{S}|\Phi)p(\Phi)} \\ &= \frac{p(\mathcal{S}, \mathcal{A}|\Theta, \Phi)p(\Theta)}{p(\mathcal{A}, \mathcal{S}|\Phi)} \end{aligned}$$

où,

- $p(A_k, S_k|\Phi)$ est une constante de normalisation.
- $p(\Theta)$ est la distribution *a priori* de Θ . En supposant que les nucléotides dans le motif sont indépendants, on a : $p(\Theta) = \prod_{j=1}^W p(\theta_j)$ où $\theta_j = (\theta_{A,j}, \theta_{C,j}, \theta_{G,j}, \theta_{T,j})$. Sachant que $p(\mathcal{S}, \mathcal{A}|\Theta, \Phi)$ est une distribution multinomiale, on suppose que $p(\Theta)$ est une distribution de Dirichlet puisque cette dernière est un "conjugate prior" pour la distribution multinomiale et on obtient : $p(\Theta) = \prod_{j=1}^W \frac{1}{B(\alpha_j)} \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{\alpha_{n,j}-1}$
- $p(\mathcal{S}, \mathcal{A}|\Theta, \Phi)$ est la distribution déterminée précédemment qu'on peut réécrire ici comme :

$$p(\mathcal{S}, \mathcal{A}|\Theta, \Phi) = C(\mathcal{S}, \mathcal{A}, \Phi) \prod_{k=1}^N \prod_{i=a_k}^{a_k+W-1} \Theta[S_k[i], i - a_k] = \prod_{k=1}^N \prod_{j=1}^W \Theta[S_k[j + a_k], j]$$

En inversant l'ordre des produits et en introduisant la fonction indicatrice $1(S_k[j + a_k] = n)$ et $c_{n,j} = \sum_{k=1}^N 1(S_k[j + a_k] = n)$ le nombre de nucléotide n à la position j sur toutes les séquences, on a :

$$\begin{aligned} p(\mathcal{S}, \mathcal{A}|\Theta, \Phi) &= C(\mathcal{S}, \mathcal{A}, \Phi) \prod_{j=1}^W \prod_{k=1}^N \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{1(S_k[j+a_k]=n)} \\ &= C(\mathcal{S}, \mathcal{A}, \Phi) \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \prod_{k=1}^N \Theta[n, j]^{1(S_k[j+a_k]=n)} \\ &= C(\mathcal{S}, \mathcal{A}, \Phi) \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{\left(\sum_{k=1}^N 1(S_k[j+a_k]=n)\right)} \\ &= C(\mathcal{S}, \mathcal{A}, \Phi) \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{c_{n,j}} \end{aligned}$$

Ainsi en injectant tout, on obtient :

$$\begin{aligned} p(\Theta|\mathcal{A}, \mathcal{S}, \Phi) &\propto \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{c_{n,j}} \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{\alpha_{n,j}-1} \\ &\propto \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{c_{n,j} + \alpha_{n,j} - 1} \end{aligned}$$

Donc $p(\Theta|\mathcal{A}, \mathcal{S}, \Phi)$ est bien une distribution de Dirichlet aussi mais de paramètres $c_j + \alpha_j$

Maintenant calculons $p(A_k|A_{-k}, \Theta, \mathcal{S}, \Phi)$:

$$\begin{aligned}
p(A_k|A_{-k}, \Theta, \mathcal{S}, \Phi) &= \frac{p(A_k, A_{-k}, \mathcal{S}|\Theta, \Phi)}{p(A_{-k}, \mathcal{S}|\Theta, \Phi)} \\
&= \frac{p(\mathcal{A}, \mathcal{S}|\Theta, \Phi)}{p(A_{-k}, \mathcal{S}|\Theta, \Phi)} \\
&\propto p(\mathcal{A}, \mathcal{S}|\Theta, \Phi) \\
&\propto \prod_{i=1}^N p(A_i, S_i|\Theta, \Phi) \\
&\propto p(A_k, S_k|\Theta, \Phi) \prod_{i \neq k} p(A_i, S_i|\Theta, \Phi) \\
&\propto p(A_k, S_k|\Theta, \Phi) C(A_i, S_i, \Theta, \Phi) \quad \forall i \neq k \\
&\propto p(A_k, S_k|\Theta, \Phi)
\end{aligned}$$

On constate que $p(A_k|A_{-k}, \Theta, \mathcal{S}, \Phi)$ ne dépend que de la séquence k .

Enfin, si Φ est une chaîne de Markov $\text{CM}(u_\Phi, P_\Phi)$ avec comme paramètre $\Phi = (u_\Phi, P_\Phi)$ la distribution initiale de la chaîne $u_\Phi \in R^4$ et sa matrice de transition $P_\Phi \in R^{4 \times 4}$, les nucléotides appartenant aux séquences de part et d'autre du motif ne sont plus indépendants.

Pour la séquence précédant le motif, cela se traduit par :

$$u_\Phi[S_k[1]] \prod_{l=2}^{a_k-1} P_\Phi[S_k[l-1], S_k[l]]$$

Pour la séquence suivant le motif, nous obtenons :

$$\prod_{l=a_k+W}^{|S_k|} P_\Phi[S_k[l-1], S_k[l]]$$

Par contre, la modélisation du motif n'est pas impactée.

Au final, nous obtenons :

$$\begin{aligned}
p(\mathcal{S}, \mathcal{A}|\Theta, \Phi) &= \prod_{k=1}^N \left(u_\Phi[S_k[1]] \prod_{l=2}^{a_k-1} P_\Phi[S_k[l-1], S_k[l]] \right) \left(\prod_{i=a_k}^{a_k+W-1} \Theta[S_k[i], i - a_k] \right) \\
&\quad \cdot \left(\prod_{l=a_k+W}^{|S_k|} P_\Phi[S_k[l-1], S_k[l]] \right) \frac{1}{|S_k| - W + 1}
\end{aligned}$$

Et ces deux dernières équations reste inchangées malgré que Φ soit une chaîne de Markov.

$$p(\Theta|\mathcal{A}, \mathcal{S}, \Phi) \propto \prod_{j=1}^W \prod_{n=\{A,C,G,T\}} \Theta[n, j]^{c_{n,j} + \alpha_{n,j} - 1}$$

$$\begin{aligned}
p(A_k|A_{-k}, \Theta, \mathcal{S}, \Phi) &\propto p(\mathcal{A}, \mathcal{S}|\Theta, \Phi) \\
&\propto p(A_k, S_k|\Theta, \Phi)
\end{aligned}$$

L'échantillonnage de $p(\Theta | A, \mathcal{S}, \Phi)$ est direct grâce à la propriété de conjugaison de la distribution de Dirichlet. En effet, pour chaque position $j = 1, \dots, W$ du motif, la distribution conditionnelle de θ_j est une loi de Dirichlet dont les paramètres sont $\alpha + n_{i,j}$, où $n_{i,j}$ désigne le nombre de fois que le nucléotide i apparaît à la position j parmi les N occurrences du motif identifiées par les positions A .

On tire donc indépendamment chaque colonne θ_j de Θ selon $\text{Dir}(\alpha + n_{1,j}, \alpha + n_{2,j}, \alpha + n_{3,j}, \alpha + n_{4,j})$, ce qui est réalisé en pratique via la fonction `numpy.random.dirichlet`.

L'échantillonnage de $p(A_k | A_{-k}, \Theta, S, \Phi)$ est un peu différent. Cette distribution étant proportionnelle à $p(S_k | A_k = a, \Theta, \Phi)$ pour $a \in \{0, \dots, |S_k| - W\}$, on calcule le score (via la log-vraisemblance) pour chaque position candidate, on soustrait le maximum pour des raisons de stabilité numérique, on exponentie et on normalise afin d'obtenir un vecteur de probabilités, puis on tire la nouvelle position A_k selon cette distribution via `numpy.random.choice`. Dans le cas où Φ est une chaîne de Markov, le calcul de $p(S_k | A_k = a, \Theta, \Phi)$ décompose la séquence en trois parties : la partie avant le motif et la partie après le motif sont évaluées selon la chaîne de Markov de paramètres (u_ϕ, P_ϕ) , tandis que la partie correspondant au motif est évaluée selon la matrice Θ , avec une attention particulière portée à la transition entre le dernier nucléotide du motif et le premier nucléotide suivant, qui est conditionnée par P_ϕ plutôt que par u_ϕ .

2.2 Implémentation

Notre implémentation suit l'algorithme de Gibbs sans balayage systématique, c'est-à-dire qu'à chaque itération, l'ordre dans lequel les positions $A_1, \dots, A_N$ sont mises à jour est tiré aléatoirement via une permutation uniforme, conformément à l'algorithme 1 de l'énoncé. Les paramètres Φ du modèle hors-motif sont estimés une seule fois, avant le début des itérations, par maximum de vraisemblance sur l'ensemble des séquences $\mathcal{S}$. Plus précisément, la distribution initiale u_ϕ est estimée comme la fréquence empirique des premiers nucléotides de chaque séquence, et la matrice de transition P_ϕ est estimée en comptant toutes les transitions entre nucléotides consécutifs dans l'ensemble des séquences, puis en normalisant chaque ligne. Cette estimation est réalisée directement sur les séquences de $\mathcal{S}$, ce qui est justifié par le fait que la longueur W du motif est petite devant les longueurs $|S_k|$, de sorte que l'impact des sous-séquences correspondant au motif sur l'estimation de Φ est négligeable. Concernant le choix des estimateurs $\hat{A}$ et $\hat{\Theta}$ renvoyés par l'algorithme, nous avons opté pour la paire (A, Θ) qui maximise la log-vraisemblance $\log p(\mathcal{S} | A, \Theta, \Phi)$ parmi toutes celles échantillonnées au cours des itérations. Afin de pallier la sensibilité de l'algorithme à l'initialisation, celui-ci est lancé plusieurs fois avec des positions initiales tirées aléatoirement de façon uniforme dans $\{0, \dots, |S_k| - W\}$ pour chaque séquence k , et la meilleure solution au sens de la log-vraisemblance est conservée parmi tous les runs. Par ailleurs, une correction de décalage de type Metropolis-Hastings est appliquée périodiquement toutes les `shift_freq` itérations : pour chaque valeur de décalage $\delta \in \{-2, -1, 1, 2\}$, on propose le vecteur $A^{\text{new}} = A + \delta$, on rééchantillonne Θ conditionnellement à A^{new} , et on accepte la proposition avec probabilité $\min(1, \exp(\log p(\mathcal{S} | A^{\text{new}}, \Theta^{\text{new}}, \Phi) - \log p(\mathcal{S} | A, \Theta, \Phi)))$, ce qui permet à l'algorithme de s'échapper des optima locaux correspondant à un décalage constant de toutes les positions par rapport à la solution optimale.

2.3 Expérimentations

Nous observons que les résultats varient d'une exécution à l'autre, et ce, indépendamment du nombre d'itérations. Ce comportement s'explique par la nature stochastique de l'algorithme : à chaque lancement, l'initialisation aléatoire des positions peut conduire l'algorithme vers un optimum local différent, dont il ne parvient pas toujours à s'échapper. C'est pourquoi il est préférable de lancer l'algorithme plusieurs fois lors de la réalisation d'expériences.

Comparaison entre Gibbs avec et sans décalage en fonction du nombre d'itération

Nous avons ensuite comparé les performances de l'algorithme de Gibbs classique et de sa variante avec gestion du décalage (période de 10), en moyennant les résultats sur 5 exécutions indépendantes pour chaque nombre d'itérations testé. Les résultats sont présentés dans la figure ci-dessous.

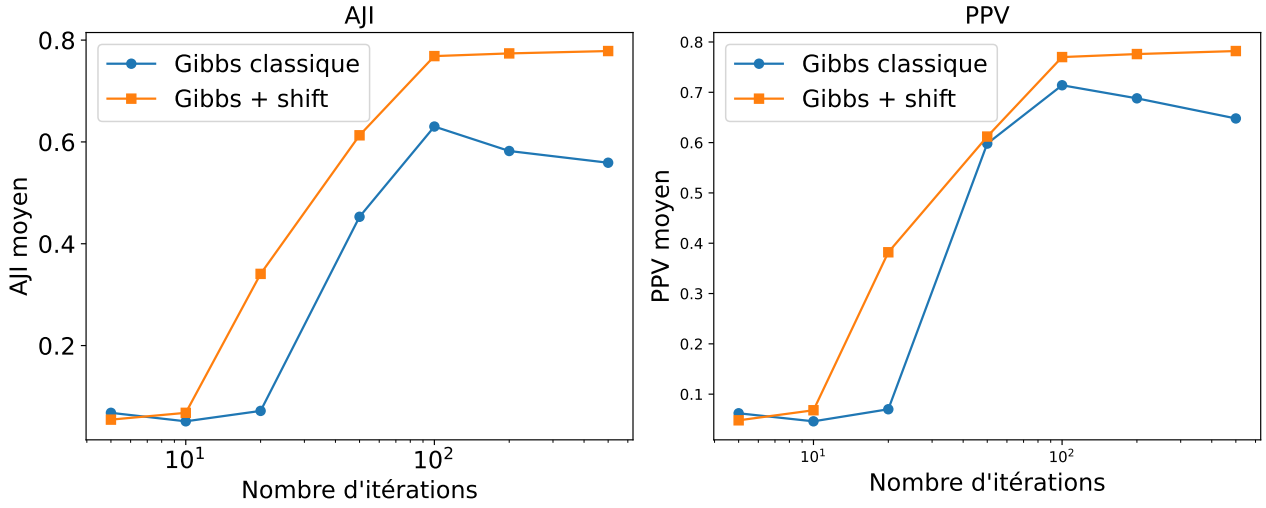


FIGURE 6 – AJI et PPV moyens (sur 5 runs) en fonction du nombre d'itérations, pour le Gibbs classique et le Gibbs avec décalage.

On constate que pour un faible nombre d'itérations, les deux algorithmes affichent des performances limitées et comparables. À partir d'une centaine d'itérations environ, les deux commencent à converger, mais vers des niveaux distincts. Le Gibbs classique se stabilise autour d'un AJI de 0,6, tandis que le Gibbs avec décalage atteint un AJI d'environ 0,8. Cet écart suggère que le Gibbs classique se piège dans un optimum local que les propositions de décalage permettent de dépasser, confirmant ainsi l'intérêt de cette extension pour explorer plus efficacement l'espace des solutions.

Analyse de l'impact de W

Le graphique ci-dessous illustre le comportement de l'algorithme lorsque la longueur W utilisée ne correspond pas à la longueur réelle du motif, fixée ici à $W^* = 10$. Ce sont les séquences artificielles qui ont été utilisées lors de cette expérience.

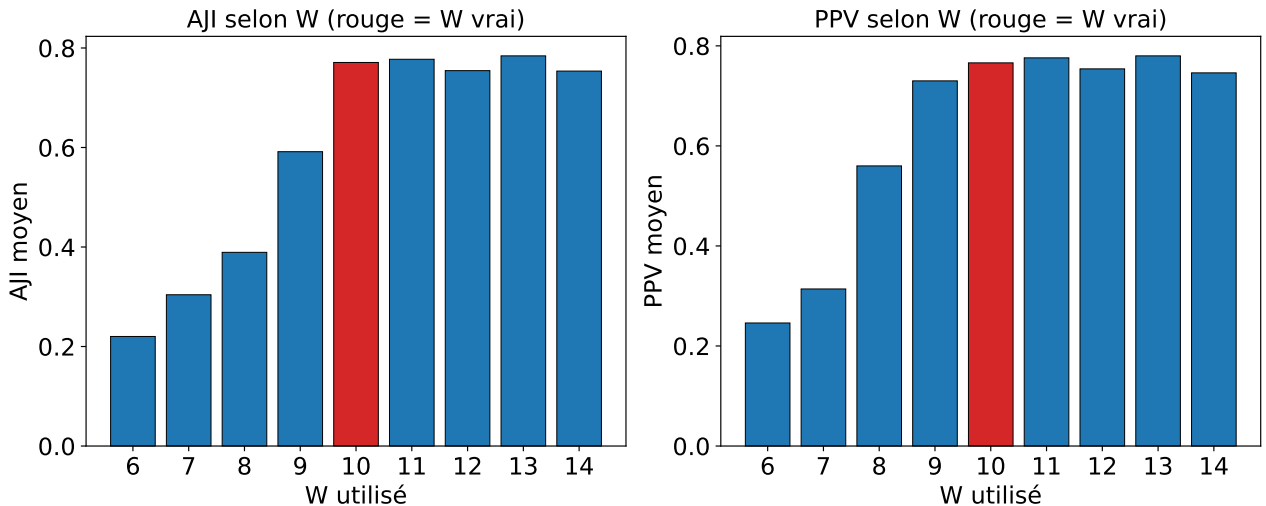


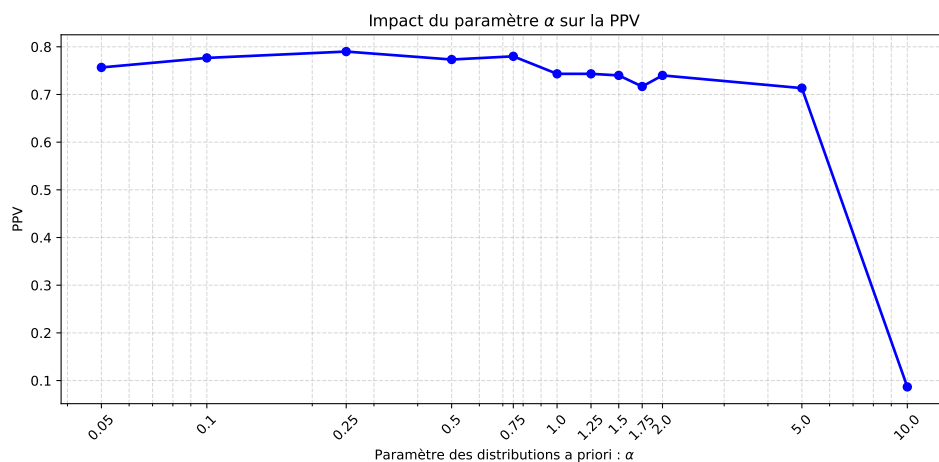
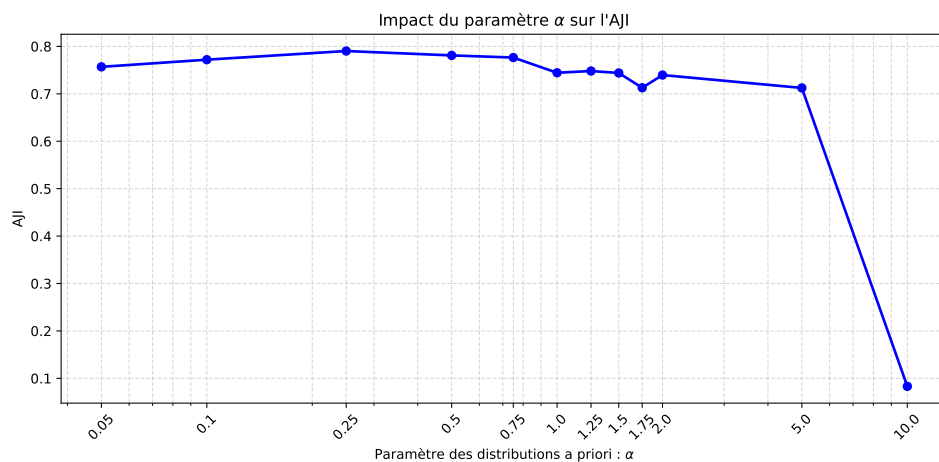
FIGURE 7 – AJI et PPV en fonction de la longueur W utilisée par l'algorithme, pour un motif de longueur réelle $W^* = 10$.

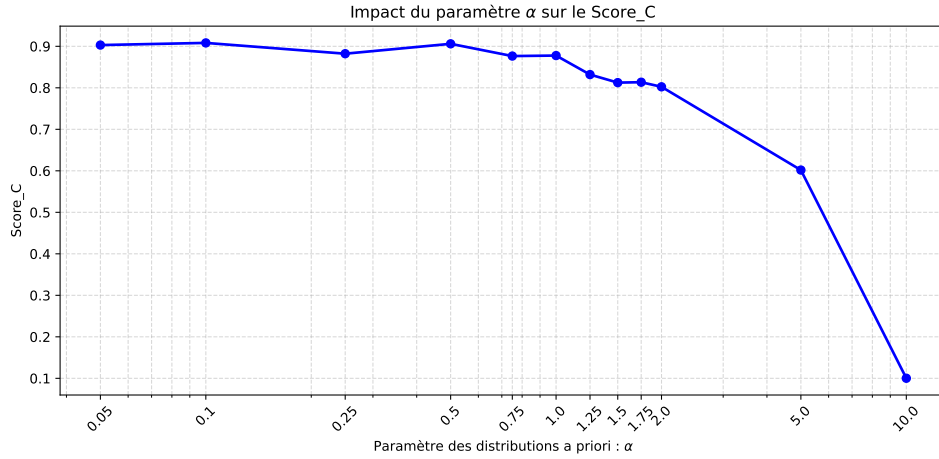
On observe que lorsque $W < W^*$, les performances se dégradent fortement : l'algorithme cherche un motif trop court et ne parvient pas à capturer l'ensemble des positions conservées, ce qui se traduit par un AJI et un PPV faibles. En revanche, lorsque $W > W^*$, les performances restent relativement

stables. Ce résultat s'explique intuitivement par le fait qu'inclure des nucléotides supplémentaires autour du vrai motif ne perturbe pas fondamentalement l'algorithme : ces positions additionnelles sont générées selon le modèle hors-motif Φ , et leur distribution multinomiale, proche de celle du fond, n'introduit pas de signal contradictoire fort (pour autant que les nucléotides du fond soient générés par une multinomiale, ce qui est bien le cas ici). En revanche, un motif trop court force l'algorithme à ignorer certaines positions, ce qui nuit directement à la qualité de la détection.

Analyse de alpha

Par après, nous avons testé plusieurs valeurs de paramètres des distributions *a priori* c'est-à-dire le paramètre α de la distribution de Dirichlet. Pour faire ces tests, nous avons utilisé les séquences artificielles avec l'algorithme de Gibbs avec le décalage. Nous précisons que chaque élément de α avait la même valeur pour ne pas favoriser arbitrairement un nucléotide. Puisque nous n'obtenons pas les mêmes résultats à chaque exécution, nous avons fait une moyenne de cinq échantillons et ainsi, nous obtenons ceci :





Globalement, le même comportement peut être observé pour chaque critère de performance. Les performances sont équivalentes pour un α inférieur ou égal à 0.75, mais nous notons la présence d'un léger maximum en 0.25. Ensuite, les performances diminuent légèrement de 0.75 à 2, avant de s'effondrer à 10. Nous en déduisons donc que le modèle tend à favoriser des motifs plus marqués avec des nucléotides qui apparaissent plus souvent à une position donnée du motif.¹ Cette dernière observation définit bien un motif, si ce n'était pas le cas il n'y aurait pas vraiment de motif défini.

Choix du modèle hors motif : Multinomiale ou chaîne de Markov

En réalisant le code et les expériences, nous nous sommes rendu compte que la multinomiale était un cas particulier de la chaîne de Markov. En effet, lors de nos tests nous avons constaté que les lignes de la matrice de transition P_Φ étaient égales si le modèle utilisé était une multinomiale. Autrement dit, la probabilité de passer d'un nucléotide à un autre était la même peu importe le nucléotide de départ. Ainsi, utiliser uniquement une multinomiale pour le modèle hors motif ne permet pas de généraliser au cas d'une chaîne de Markov mais l'inverse est correcte.

2.4 Expériences supplémentaires

Faute de temps, nous n'avons pas pu produire de graphiques systématiques pour les expériences suivantes, mais des tests manuels sur des séquences générées par nos soins nous ont permis de dégager plusieurs constats qualitatifs.

Premièrement, lorsque la longueur des séquences L devient très élevée (de l'ordre de 1000 nucléotides ou plus) pour un motif de longueur $W = 10$, les performances de l'algorithme se dégradent progressivement. Ce phénomène s'explique par le fait qu'une séquence longue contient davantage de sous-séquences hors-motif, et que parmi celles-ci, certaines peuvent par pur hasard ressembler fortement au motif recherché. Le modèle de fond Φ peut en effet générer localement des enchaînements de nucléotides proches du motif, ce qui crée des faux maxima locaux dans la distribution conditionnelle $p(A_k | A_{-k}, \Theta, S, \Phi)$ et perturbe la convergence de l'algorithme vers les vraies positions.

Deuxièmement, un phénomène analogue est observé lorsque la longueur W du motif est trop petite. Un motif court est en effet statistiquement plus susceptible d'apparaître par hasard dans les séquences hors-motif, ce qui rend sa détection plus ambiguë. Plus W est petit, plus le signal apporté par le motif est dilué dans le bruit de fond, et plus l'algorithme a de difficultés à distinguer les vraies occurrences des coïncidences fortuites.

Enfin, lors de la génération de séquences homogènes, c'est-à-dire dont les nucléotides hors-motif sont tirés selon une distribution uniforme, nous avons constaté que l'algorithme détecte plus aisément

1. https://en.wikipedia.org/wiki/Dirichlet_distribution

les motifs dont certaines positions présentent un nucléotide fortement dominant, c'est-à-dire dont la probabilité associée dans Θ est proche de 1. Ce résultat est intuitif : lorsque le fond est uniforme, un nucléotide très probable à une position donnée du motif crée un contraste marqué entre la vraisemblance du motif et celle du fond, ce qui facilite l'identification des vraies occurrences par l'algorithme. À l'inverse, un motif dont les colonnes de Θ sont peu discriminantes, c'est-à-dire proches d'une distribution uniforme, est difficile à distinguer du fond, même lorsque ce dernier est lui-même uniforme.

2.5 Compétition

Concernant la compétition sur les séquences artificielles, nous avons atteint la troisième place avec un AJI et un PPV de 0.80. Notre algorithme est parvenu à déterminer automatiquement la bonne longueur de motif $W = 12$ via la fonction `find_best_W`, qui sélectionne la valeur maximisant la log-vraisemblance sur plusieurs runs. Ce résultat valide l'approche adoptée pour la recherche automatique de W dans un intervalle donné.

Concernant la compétition sur les séquences réelles, nous avons également atteint la troisième place, avec un AJI de 0.70 et un PPV de 0.89. Notre algorithme a estimé une longueur de motif $W = 28$, légèrement supérieure à la longueur réelle moyenne des motifs, qui est d'environ 25. Comme mis en évidence dans la section expérimentale consacrée à l'impact de W , une surestimation modérée de la longueur du motif est moins pénalisante qu'une sous-estimation : les positions supplémentaires incluses dans le motif correspondent à des nucléotides sûrement peu conservés, proches de la distribution hors-motif Φ , et ne perturbent donc pas fondamentalement la détection. Cela explique que nous ayons tout de même obtenu des scores satisfaisants malgré cette légère surestimation de W . Par ailleurs, le PPV nettement plus élevé que l'AJI sur les séquences réelles suggère que, lorsque l'algorithme identifie correctement une occurrence du motif, il le fait avec une bonne précision, mais que le décalage introduit par la surestimation de W pénalise davantage l'AJI, qui est plus sensible au recouvrement exact des positions prédites et réelles.

2.6 Contribution de chacun

Nous avons tous les deux contribué à hauteur de 50% du travail effectué pour réaliser ce projet.

2.7 Utilisation de l'intelligence artificielle

Nous avons mobilisé l'intelligence artificielle **Gemini** pour la correction orthographique et grammaticale de ce rapport, ainsi que pour la génération de graphiques et de logs de console dans la partie expérimentale. Cet outil a également servi de support documentaire pour approfondir certains aspects théoriques du sujet de manière générale.

Une part de son utilisation a concerné l'optimisation de notre code **Python**. Initialement, la vitesse de traitement ne dépassait pas quelques itérations par seconde. En nous appuyant sur des exemples d'optimisation de boucles et d'opérations matricielles fournis par l'IA, nous avons pu vectoriser les sections critiques de notre algorithme, notamment pour le calcul de la log-vraisemblance et l'échantillonnage des A_k . Ces ajustements ont permis d'accroître la vitesse d'exécution d'un facteur de 50 sans avoir modifié la logique de notre implémentation initiale.

En somme, nous avons fait usage de cet outil dans le respect des consignes qui nous ont été transmises.